Verilog RTL Design and RISC-V Processor Development

Day 1: RTL Building Blocks and Self-Checking Verification

Core Activities 1.1 to 1.4: Design files, simulation output, waveforms and tech notes

Tools: Icarus Verilog 12.0 (iverilog -g2012, vvp); waveforms from VCD dumps (viewable in GTKWave). Every testbench contains an independent reference model, directed corner cases plus random stimulus, automatic comparison, and a PASS/FAIL summary.

Activity	Result
1.1 Parameterized ALU	5008 tests, 0 errors -> PASS
1.2 Up/Down Counter	350 cycles, 0 errors -> PASS
1.3 Barrel Shifter	1728 tests, 0 errors -> PASS
1.4 Register File	4069 checks, 0 errors -> PASS

1.1 Parameterized ALU

Design approach. Combinational module with parameter WIDTH (default 8). A 4-bit op selects ADD, SUB, AND, OR, XOR, NOR, SLL, SRL, SRA, SLT, SLTU. Outputs: result y, zero, carry, overflow. Add/sub use a WIDTH+1 bit temporary to capture carry/borrow. Signed overflow for ADD: operands share a sign but the result sign differs; for SUB: operands differ in sign and the result sign differs from a. Shift amount is the low log2(WIDTH) bits of b. SRA uses the signed arithmetic shift operator.

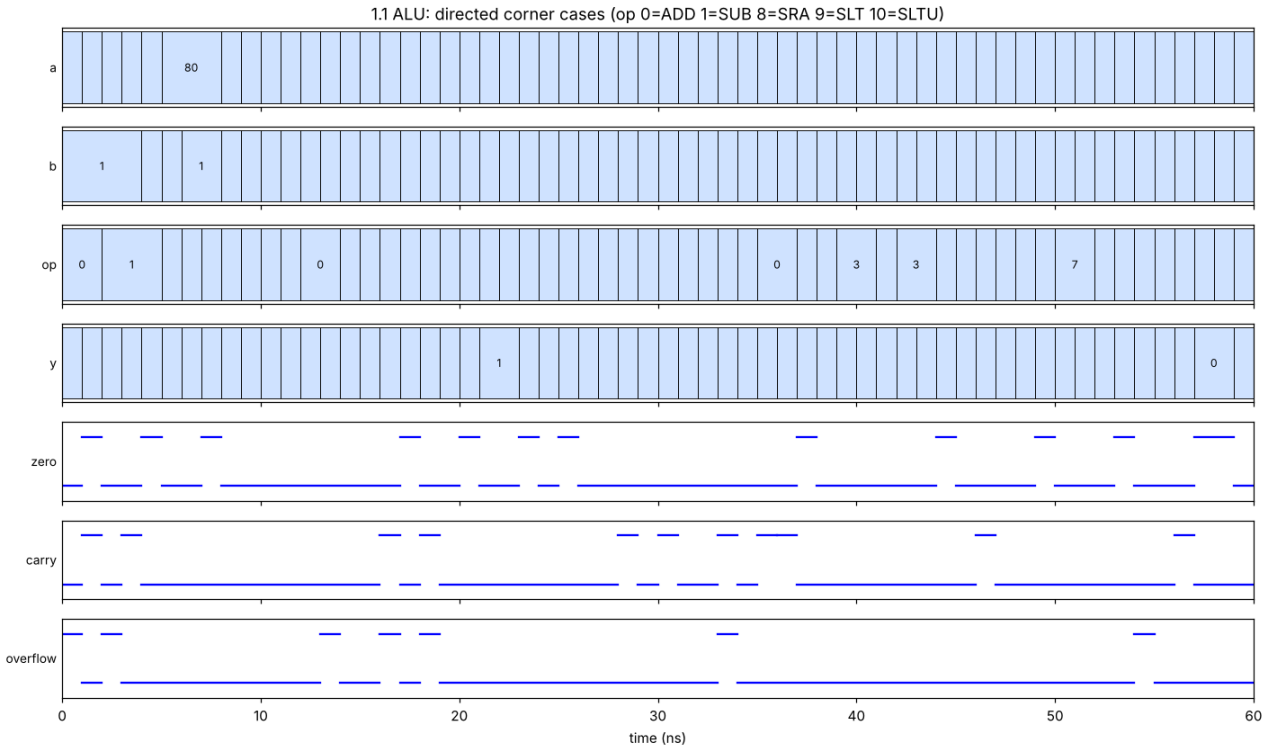
Verification. 8 directed corner cases (signed overflow, carry with zero result, borrow, SRA sign extension, SLT vs SLTU on 0x80/0x01) plus 5000 random (a, b, op) vectors. y, zero, carry and overflow are all compared against a reference model inside the testbench.

Issues / notes. SUB carry is a borrow flag (1 when a < b unsigned). Flags are meaningful only for ADD/SUB.

Simulation output

```
ALU TEST: 5008 tests, 0 errors -> PASS
```

Waveform



1.1 RTL source (alu.v)

```
// 1.1 Parameterized ALU
module alu #(parameter WIDTH = 8) (
    input  wire [WIDTH-1:0] a, b,
    input  wire [3:0]       op,
    output reg  [WIDTH-1:0] y,
    output wire             zero,
    output reg              carry,
    output reg              overflow
);
    localparam ADD=4'd0, SUB=4'd1, AND_=4'd2, OR_=4'd3, XOR_=4'd4, NOR_=4'd5,
               SLL=4'd6, SRL=4'd7, SRA=4'd8, SLT=4'd9, SLTU=4'd10;
    localparam SW = $clog2(WIDTH);
    wire [SW-1:0] sh = b[SW-1:0];
    reg [WIDTH:0] t;
    assign zero = (y == {WIDTH{1'b0}});
    always @* begin
        t = 0; carry = 0; overflow = 0;
        case (op)
            ADD: begin t = {1'b0,a} + {1'b0,b}; y = t[WIDTH-1:0]; carry = t[WIDTH];
                       overflow = (a[WIDTH-1]==b[WIDTH-1]) && (y[WIDTH-1]!=a[WIDTH-1]); end
            SUB: begin t = {1'b0,a} - {1'b0,b}; y = t[WIDTH-1:0]; carry = t[WIDTH]; // carry = borrow
                       overflow = (a[WIDTH-1]!=b[WIDTH-1]) && (y[WIDTH-1]!=a[WIDTH-1]); end
            AND_: y = a & b;
            OR_:  y = a | b;
            XOR_: y = a ^ b;
            NOR_: y = ~(a | b);
            SLL:  y = a << sh;
            SRL:  y = a >> sh;
            SRA:  y = $signed(a) >>> sh;
            SLT:  y = {{(WIDTH-1){1'b0}}, ($signed(a) < $signed(b))};
            SLTU: y = {{(WIDTH-1){1'b0}}, (a < b)};
            default: y = {WIDTH{1'b0}};
        endcase
    end
endmodule
```

1.1 Testbench (alu_tb.v)

```
`timescale 1ns/1ps
module alu_tb;
    parameter W = 8;
    reg  [W-1:0] a, b; reg [3:0] op;
    wire [W-1:0] y; wire zero, carry, overflow;
    alu #(W) dut(.a(a),.b(b),.op(op),.y(y),.zero(zero),.carry(carry),.overflow(overflow));
    integer errors = 0, tests = 0, i;
    reg [W-1:0] exp_y; reg exp_c, exp_v; reg [W:0] t;
    reg signed [W-1:0] sa, sb;
    task check;
        begin
            sa = a; sb = b; exp_c = 0; exp_v = 0;
            case (op)
                0: begin t = a + b; exp_y = t[W-1:0]; exp_c = t[W];
                         exp_v = (a[W-1]==b[W-1]) && (exp_y[W-1]!=a[W-1]); end
                1: begin t = {1'b0,a} - {1'b0,b}; exp_y = t[W-1:0]; exp_c = t[W];
                         exp_v = (a[W-1]!=b[W-1]) && (exp_y[W-1]!=a[W-1]); end
                2: exp_y = a & b;   3: exp_y = a | b;
                4: exp_y = a ^ b;   5: exp_y = ~(a | b);
                6: exp_y = a << (b % W);
                7: exp_y = a >> (b % W);
                8: exp_y = sa >>> (b % W);
                9: exp_y = (sa < sb);
                10: exp_y = (a < b);
                default: exp_y = 0;
            endcase
            #1; tests = tests + 1;
            if (y !== exp_y || zero !== (exp_y==0) || carry !== exp_c || overflow !== exp_v) begin
                errors = errors + 1;
                $display("FAIL op=%0d a=%h b=%h : y=%h(exp %h) z=%b c=%b(exp %b) v=%b(exp %b)",
                         op,a,b,y,exp_y,zero,carry,exp_c,overflow,exp_v);
            end
        end
    endtask
    initial begin
        $dumpfile("alu.vcd"); $dumpvars(0, alu_tb);
        // directed corner cases
        a=8'h7F; b=8'h01; op=0; check;   // signed overflow
        a=8'hFF; b=8'h01; op=0; check;   // carry, zero
        a=8'h80; b=8'h01; op=1; check;   // sub overflow
        a=8'h00; b=8'h01; op=1; check;   // borrow
        a=8'h55; b=8'h55; op=1; check;   // zero
        a=8'h80; b=8'h02; op=8; check;   // SRA sign
        a=8'h80; b=8'h01; op=9; check;   // SLT
        a=8'h80; b=8'h01; op=10; check;  // SLTU
        // exhaustive-ish random
        for (i = 0; i < 5000; i = i + 1) begin
            a = $random; b = $random; op = $urandom % 11; check;
        end
        $display("ALU TEST: %0d tests, %0d errors -> %s", tests, errors, errors==0 ? "PASS" : "FAIL");
        $finish;
    end
endmodule
```

1.2 Up/Down Counter

Design approach. Synchronous counter with asynchronous active-low reset. Inputs en, up, load, load_val. Priority: reset > load > enable. wrap is a combinational flag, high when an enabled count would roll over (all-ones going up, zero going down).

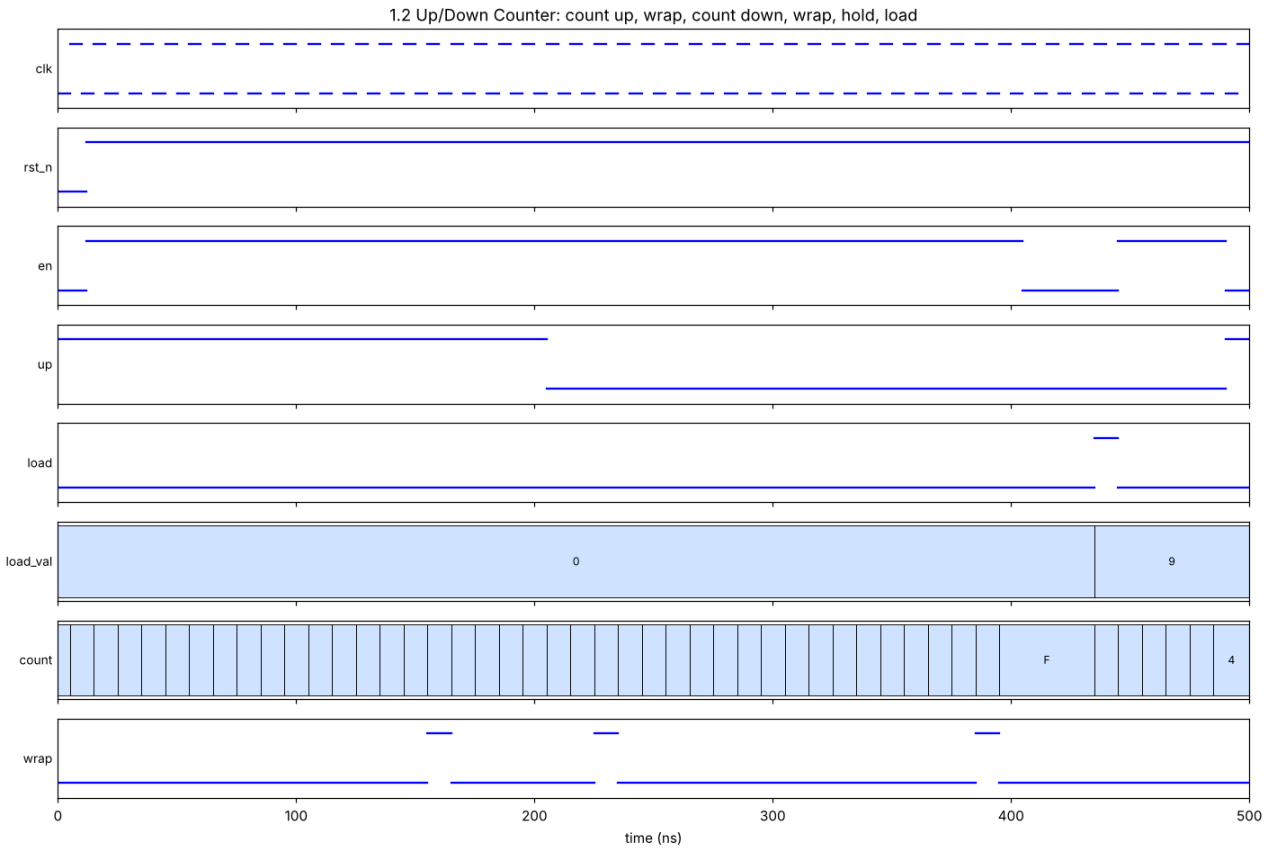
Verification. Directed sequence: up through wrap, down through wrap, hold, load 9 then count down; then 300 cycles of random en/up/load/load_val with random asynchronous resets. A cycle-based reference model is compared every clock, including wrap.

Issues faced. (1) First testbench had a race: stimulus and checker both ran on the negedge, so wrap was compared against stale inputs. Fixed by sampling at the posedge. (2) The model first used a synchronous reset and produced false failures on async reset pulses; it now matches the DUT. (3) The first clock edge is skipped in checking because the DUT output is X until reset takes effect.

Simulation output

```
COUNTER TEST: 350 cycles, 0 errors -> PASS
```

Waveform



1.2 RTL source (updown_counter.v)

```
// 1.2 Up/Down Counter with synchronous load, enable, wrap flag
module updown_counter #(parameter WIDTH = 4) (
    input  wire             clk, rst_n, en, up, load,
    input  wire [WIDTH-1:0] load_val,
    output reg  [WIDTH-1:0] count,
    output wire             wrap      // high when next count wraps around
);
    assign wrap = en & ~load & (up ? (count == {WIDTH{1'b1}}) : (count == 0));
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)       count <= 0;
        else if (load)    count <= load_val;
        else if (en)      count <= up ? count + 1'b1 : count - 1'b1;
    end
endmodule
```

1.2 Testbench (updown_counter_tb.v)

```
`timescale 1ns/1ps
module updown_counter_tb;
    parameter W = 4;
    reg clk=0, rst_n=0, en=0, up=1, load=0; reg [W-1:0] load_val=0;
    wire [W-1:0] count; wire wrap;
    updown_counter #(W) dut(clk,rst_n,en,up,load,load_val,count,wrap);
    always #5 clk = ~clk;
    reg [W-1:0] model = 0; reg exp_wrap;
    integer errors=0, i, cycles=0;
    // Reference model + checker: both sample pre-edge values at posedge
    // (nonblocking updates land after this block reads), avoiding stimulus races.
    always @(posedge clk) begin
        cycles = cycles + 1;
        exp_wrap = en & ~load & (up ? (model=={W{1'b1}}) : (model==0));
        if (cycles > 1 && (count !== model || wrap !== exp_wrap)) begin
            errors = errors + 1;
            $display("FAIL t=%0t count=%0d exp=%0d wrap=%b exp=%b", $time,count,model,wrap,exp_wrap);
        end
    end
    // Model mirrors the DUT's asynchronous active-low reset
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) model <= 0;
        else if (load) model <= load_val;
        else if (en) model <= up ? model+1 : model-1;
    end
    initial begin
        $dumpfile("counter.vcd"); $dumpvars(0, updown_counter_tb);
        #12 rst_n = 1;
        en=1; up=1; repeat (20) @(posedge clk);          // up through wrap
        up=0;       repeat (20) @(posedge clk);          // down through wrap
        en=0;       repeat (3)  @(posedge clk);          // hold
        load=1; load_val=4'd9; @(posedge clk); load=0;   // load
        en=1; up=0; repeat (4) @(posedge clk);
        for (i=0;i<300;i=i+1) begin                      // random stimulus
            @(negedge clk); en=$random; up=$random; load=($urandom%10==0); load_val=$random;
            if ($urandom%97==0) rst_n=0; else rst_n=1;
        end
        @(negedge clk); @(negedge clk);
        $display("COUNTER TEST: %0d cycles, %0d errors -> %s", cycles, errors, errors==0?"PASS":"FAIL");
        $finish;
    end
endmodule
```

1.3 Barrel Shifter

Design approach. Combinational, log2(WIDTH) mux stages built with generate; stage i shifts by 2^i when shamt[i] is set. mode: 00 SLL, 01 SRL, 10 SRA (MSB fill), 11 ROL. Any shift completes in 3 mux levels for WIDTH=8.

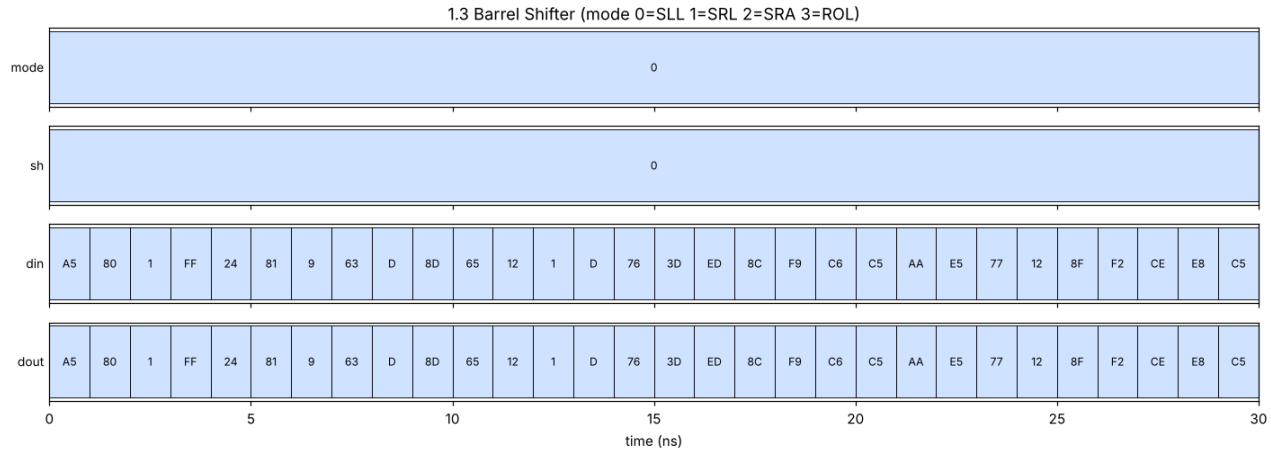
Verification. All 4 modes x all 8 shift amounts, each with 4 fixed patterns plus 50 random inputs, against operator-based reference (rotate reference uses a doubled word).

Issues. None functional. The rotate reference needs a special case for shift 0.

Simulation output

```
SHIFTER TEST: 1728 tests, 0 errors -> PASS
```

Waveform



1.3 RTL source (barrel_shifter.v)

```
// 1.3 Barrel Shifter: log2(WIDTH)-stage mux network, combinational
// mode: 00=SLL 01=SRL 10=SRA 11=ROL
module barrel_shifter #(parameter WIDTH = 8) (
    input  wire [WIDTH-1:0]         din,
    input  wire [$clog2(WIDTH)-1:0] shamt,
    input  wire [1:0]               mode,
    output wire [WIDTH-1:0]         dout
);
    localparam N = $clog2(WIDTH);
    wire fill = (mode == 2'b10) & din[WIDTH-1];
    wire [WIDTH-1:0] stage [0:N];
    assign stage[0] = din;
    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin : g
            localparam S = 1 << i;
            wire [WIDTH-1:0] sh_l = {stage[i][WIDTH-1-S:0], {S{1'b0}}};
            wire [WIDTH-1:0] sh_r = {{S{fill}}, stage[i][WIDTH-1:S]};
            wire [WIDTH-1:0] rot  = {stage[i][WIDTH-1-S:0], stage[i][WIDTH-1:WIDTH-S]};
            wire [WIDTH-1:0] moved = (mode==2'b00) ? sh_l : (mode==2'b11) ? rot : sh_r;
            assign stage[i+1] = shamt[i] ? moved : stage[i];
        end
    endgenerate
    assign dout = stage[N];
endmodule
```

1.3 Testbench (barrel_shifter_tb.v)

```
`timescale 1ns/1ps
module barrel_shifter_tb;
    parameter W = 8;
    reg [W-1:0] din; reg [$clog2(W)-1:0] sh; reg [1:0] mode;
    wire [W-1:0] dout;
    barrel_shifter #(W) dut(din,sh,mode,dout);
    reg [W-1:0] exp; reg signed [W-1:0] sd; reg [2*W-1:0] dbl;
    integer errors=0, tests=0, i, m, s;
    task check; begin
        sd = din; dbl = {din,din};
        case (mode)
            0: exp = din << sh;
            1: exp = din >> sh;
            2: exp = sd >>> sh;
            3: exp = (sh==0) ? din : (dbl >> (W-sh));
        endcase
        #1; tests=tests+1;
        if (dout !== exp) begin errors=errors+1;
            $display("FAIL mode=%0d din=%b sh=%0d dout=%b exp=%b",mode,din,sh,dout,exp); end
    end endtask
    initial begin
        $dumpfile("shifter.vcd"); $dumpvars(0, barrel_shifter_tb);
        for (m=0;m<4;m=m+1) for (s=0;s<W;s=s+1) begin   // all modes x all shifts
            mode=m; sh=s;
            din=8'hA5; check; din=8'h80; check; din=8'h01; check; din=8'hFF; check;
            for (i=0;i<50;i=i+1) begin din=$random; check; end
        end
        $display("SHIFTER TEST: %0d tests, %0d errors -> %s",tests,errors,errors==0?"PASS":"FAIL");
        $finish;
    end
endmodule
```

1.4 Register File

Design approach. 32 x 32 array (parameterized), two asynchronous read ports, one synchronous write port with we. Register x0 is hard-wired to zero as in RISC-V. A write-through bypass returns wdata when a port reads the register being written in the same cycle.

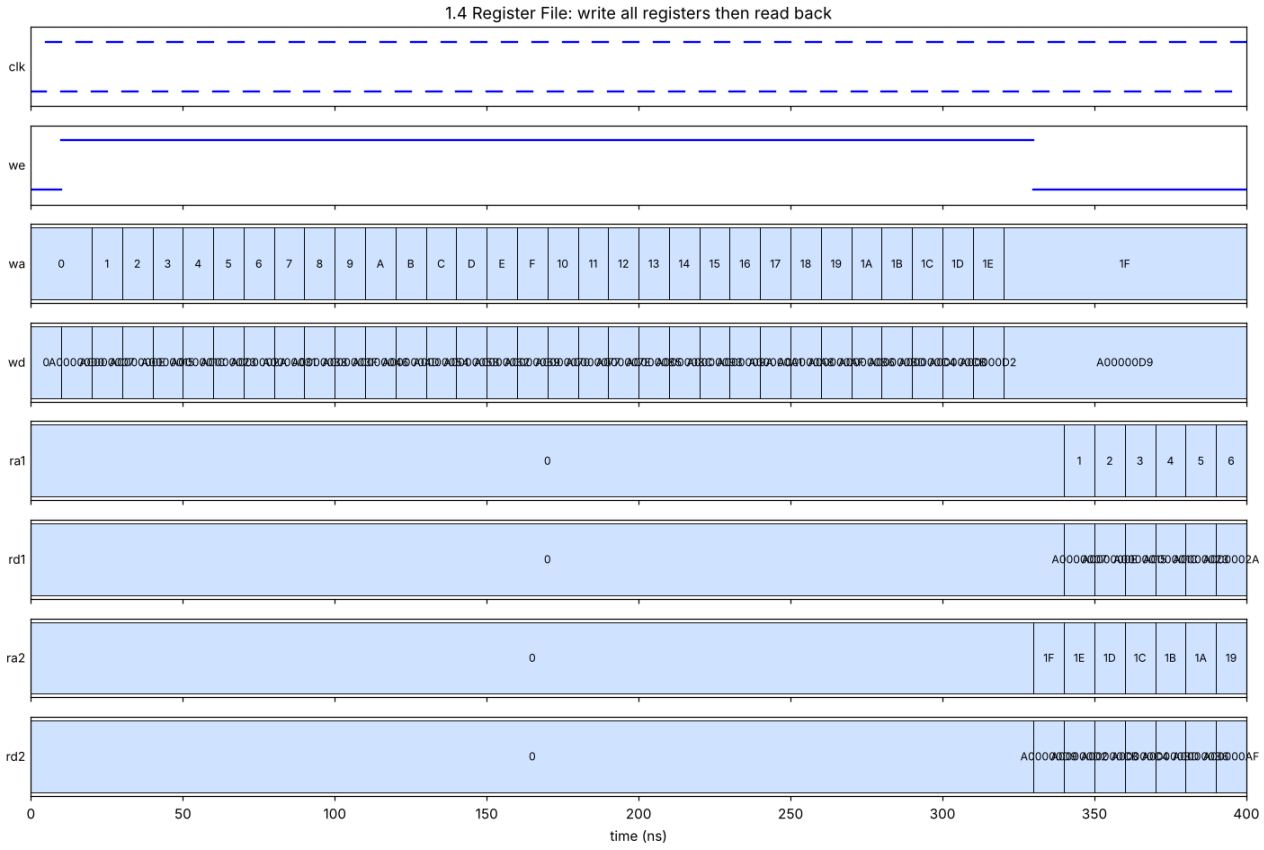
Verification. Unique data written to all 32 registers and read back on both ports; x0 write attempt; bypass test; we=0 must not write; 2000 random cycles against an array model.

Issues / notes. The bypass is a design choice; remove the (we && raddr==waddr) term if a read-old-value behaviour is required.

Simulation output

```
REGFILE TEST: 4069 checks, 0 errors -> PASS
```

Waveform



1.4 RTL source (regfile.v)

```
// 1.4 Register File: 32 x 32, 2 async read ports, 1 sync write port, x0 = 0
module regfile #(parameter WIDTH = 32, parameter DEPTH = 32) (
    input  wire                    clk, we,
    input  wire [$clog2(DEPTH)-1:0] raddr1, raddr2, waddr,
    input  wire [WIDTH-1:0]         wdata,
    output wire [WIDTH-1:0]         rdata1, rdata2
);
    reg [WIDTH-1:0] mem [0:DEPTH-1];
    integer k;
    initial for (k=0;k<DEPTH;k=k+1) mem[k] = 0;
    always @(posedge clk) if (we && waddr != 0) mem[waddr] <= wdata;
    // write-through bypass: a same-cycle read of the written register sees new data
    assign rdata1 = (raddr1 == 0) ? 0 : (we && raddr1==waddr) ? wdata : mem[raddr1];
    assign rdata2 = (raddr2 == 0) ? 0 : (we && raddr2==waddr) ? wdata : mem[raddr2];
endmodule
```

1.4 Testbench (regfile_tb.v)

```
`timescale 1ns/1ps
module regfile_tb;
    reg clk=0, we=0; reg [4:0] ra1=0, ra2=0, wa=0; reg [31:0] wd=0;
    wire [31:0] rd1, rd2;
    regfile dut(clk,we,ra1,ra2,wa,wd,rd1,rd2);
    always #5 clk=~clk;
    reg [31:0] model [0:31];
    integer errors=0, tests=0, i;
    task chk; input [31:0] got, exp; begin
        tests=tests+1;
        if (got!==exp) begin errors=errors+1; $display("FAIL t=%0t got=%h exp=%h",$time,got,exp); end
    end endtask
    initial begin
        $dumpfile("regfile.vcd"); $dumpvars(0, regfile_tb);
        for (i=0;i<32;i=i+1) model[i]=0;
        // write every register with unique data, then read back
        for (i=0;i<32;i=i+1) begin @(negedge clk); we=1; wa=i; wd=32'hA000_0000+i*7;
            if (i!=0) model[i]=wd; end
        @(negedge clk); we=0;
        for (i=0;i<32;i=i+1) begin ra1=i; ra2=31-i; #1; chk(rd1,model[i]); chk(rd2,model[31-i]); @(negedge clk); end
        // x0 must stay zero
        we=1; wa=0; wd=32'hDEADBEEF; ra1=0; #1; chk(rd1,0); @(negedge clk); we=0; ra1=0; #1; chk(rd1,0);
        // write-through bypass
        @(negedge clk); we=1; wa=5'd10; wd=32'h1234_5678; ra1=10; ra2=10; #1; chk(rd1,wd); chk(rd2,wd);
        @(negedge clk); model[10]=32'h1234_5678;
        // we=0 must not write
        we=0; wa=5'd11; wd=32'hFFFF_FFFF; @(negedge clk); ra1=11; #1; chk(rd1,model[11]);
        // random traffic
        for (i=0;i<2000;i=i+1) begin
            @(negedge clk); we=$random; wa=$random; wd=$random; ra1=$random; ra2=$random;
            #1;
            chk(rd1, (ra1==0)?0 : (we&&ra1==wa)? wd : model[ra1]);
            chk(rd2, (ra2==0)?0 : (we&&ra2==wa)? wd : model[ra2]);
            @(posedge clk); #1; if (we && wa!=0) model[wa]=wd;
        end
        $display("REGFILE TEST: %0d checks, %0d errors -> %s",tests,errors,errors==0?"PASS":"FAIL");
        $finish;
    end
endmodule
```